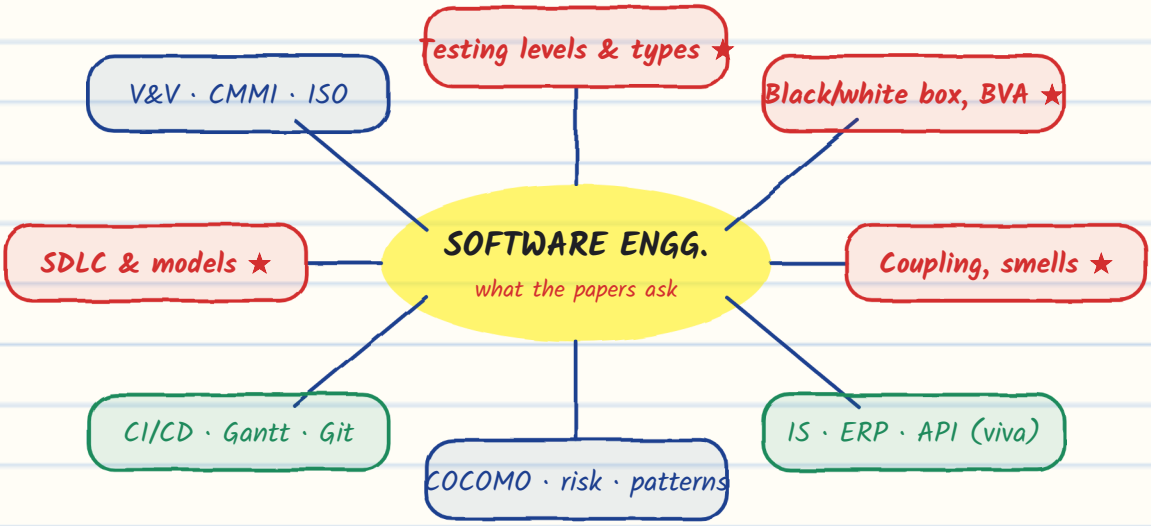
Solved Past Questions: Software Engineering

45 questions from BCS, bank, Petrobangla and BUET-set papers, solved in 12 sheets



↑ the asked topics on one page: red = asked most often, blue = written once, green = MCQ / viva

→ Papers covered

- 38th BCS Written (2018) · 49th BCS ICT paper 2, MCQ (2025)
- Bangladesh Bank AME (BIBM, 2023) · bank Assistant Programmer
- Petrobangla AM (IT) 2022 (BUET), AM (CSE) 2024 · BOGMC AM 2024
- Islamic Bank PLC SQA 2025 · Bangladesh Satellite Co. AE 2025, 2026 · bank IT viva boards

Most asked

① name the testing type
② black-box vs white-box
③ EC + BVA test cases
④ SDLC phases (10 marks)
⑤ coupling & cohesion
⑥ COCOMO, 32 KLOC

Legend yellow = key term · red circle = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked answer · **blue frame** = the question as asked

→ Contents

✓ Testing levels & types ..... p. 2
✓ Black-box, white-box, V(G) ... p. 3
✓ EC + BVA · V&V (CMMI) ..... p. 4
✓ SDLC, implementation, models ... p. 5
✓ Scheduling · CI/CD · Git ..... p. 6
✓ Coupling, cohesion, smells ... p. 7
✓ Architecture, patterns, UI ..... p. 8
✓ COCOMO & risk analysis ..... p. 9
✓ Viva rapid-fire + MCQs asked ... p. 10
✓ Night-before sheet ..... p. 11
✓ Practice MCQs + written ..... p. 12

1 Levels of Software Testing

viva ×6

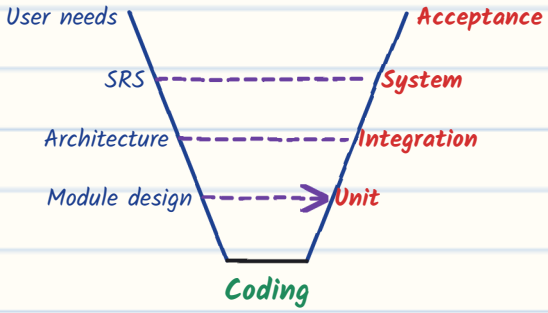
Viva What are the levels of software testing? What is unit / integration / system / acceptance testing?

bank IT viva boards (asked one by one and again as "levels")

Level	What is tested	By · against
Unit	one function / class alone (stubs, drivers, mocks)	developer · module design
Integration	interfaces and data flow between modules	developer, tester · architecture
System	whole system, functional **and** non-functional	independent testers · SRS
Acceptance	fit for business use: UAT, alpha, beta	client, users · business needs

Integration strategies **big-bang** (all at once) · **top-down** (needs **stubs** for lower modules) · **bottom-up** (needs **drivers** for upper modules) · **sandwich** (both). **Alpha** = users test at the developer's site; **beta** = real users at their own site.

Memory trick: V-model each test level checks what the document opposite it promised: module design ↔ unit, architecture ↔ integration, SRS ↔ system, user needs ↔ acceptance.



V-model: left arm builds, right arm tests

2 Name the Testing Type: Functional or Not?

written ×3

Q4(b) Identify the testing type in each scenario and say whether it is Functional, Non-Functional or Both: (1) verifying the smallest testable unit; (2) re-running tests to check the application still works after code changes; (3) performance and responsiveness under a specific amount of traffic; (4) preliminary testing of basic functions before detailed testing; (5) verifying that integrated modules work together.

Petrobangla AM (CSE) · 31.05.2024 · BOGMC (Petrobangla) AM · 2024 · also set in another recruitment paper

#	Testing type	Why	Category
1	**Unit**	smallest testable part, in isolation	Functional
2	**Regression**	re-runs the existing suite after a change	**Both**
3	**Load** (performance)	behaviour under the expected traffic	Non-functional
4	**Smoke** (build check)	shallow check of a new build's key functions	Functional
5	**Integration**	interfaces between modules	Functional

Functional vs non-functional **Functional** = **what** the system does (input → expected output). **Non-functional** = **how well**: performance, load, stress, security, usability, reliability.

Exam trap Regression re-runs functional **and** non-functional tests (a release can also get slower), so "Both". Re-testing re-runs only the failed case. Smoke = broad, shallow; sanity = narrow, deep.

3 Black-box vs White-box Testing

written + viva

Q6 Differentiate between Black Box Testing and White Box Testing.

Islamic Bank PLC, SQA Senior Officer (Network) · 14.03.2025 · viva: "What are software testing methods?"

Point	Black-box	White-box
Code knowledge	none: inputs and outputs only	full: internal logic and paths
Tests based on	requirements, SRS	source code, design
Also called	behavioural, closed-box	structural, glass-box, clear-box
Done by · level	testers, users · system, acceptance	developers · unit, integration
Techniques	EP, BVA, decision table, state transition	statement/branch/path coverage, basis path
Finds	missing or wrong functions, UI errors	logic errors, dead code, unused variables



Grey-box **partial** knowledge of the inside (DB schema, architecture, API docs) while testing from outside. Used in integration, web and penetration testing. Methods = black, white, grey box.

Memory trick **Black**: you can't see in → test the **B**ehaviour. **White**: glass walls → test the **W**iring (code paths).

4 White-box = Structural Testing

MCQ ×2

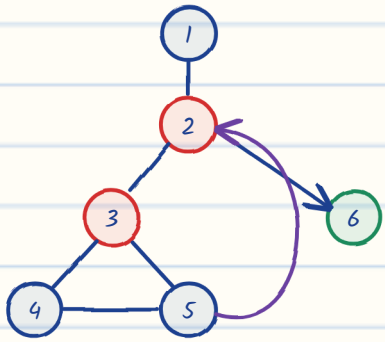
Asked MCQs ✍️ **Q86** Which technique is called structural testing? → **White box** **Q21** Which is **not** a focus of white-box testing: code coverage / data flow / user perspective / code logic? → **User perspective** (that is black-box). 49th BCS ICT paper 2 · 2025

Coverage, weakest → strongest: statement → branch (decision) → condition → MC/DC → **path**. 100% branch ⇒ 100% statement, not the reverse.

Cyclomatic complexity $V(G) = E - N + 2$ = decisions + 1 = regions = number of basis paths = minimum test cases for basis-path testing.

```
int f(int x){ int s = 0;
  while (x > 0) {       // decision 1
    if (x % 2) s += x;  // decision 2
    x--; }
  return s; }
```

Tests x = 0 (loop skipped), x = 1 (odd branch), x = 2 (even, then odd) cover every edge: **3 cases = V(G)**.



N = 6, E = 7

$V(G) = 7 - 6 + 2 = 3$

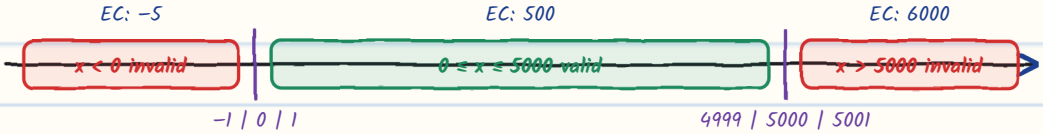
1 s=0 · 2 while · 3 if · 4 s+=x · 5 x-- · 6 return

5 Equivalence Classes & Boundary Values written

Q3 What is black box testing? A program computes the square root of an input integer between 0 and 5000. Determine the equivalence class test cases, and the test cases using boundary value analysis.

Petrobangla AM (IT) · 16.09.2022 (BUET)

Black-box testing tests the program only through its inputs and outputs, against the specification, without looking at the code (behavioural testing).



Equivalence classes: one representative per class → {−5, 500, 6000}. Every value in a class should behave alike.

Boundary values: errors cluster at the edges 0 and 5000 → {−1, 0, 5000, 5001}; the robust set adds 1, 4999 and a nominal 2500.

Input	Expected output
−1, 5001	rejected: "invalid input"
0 · 1 · 2500	0 · 1 · 50
4999 · 5000	70.70 · 70.71

Exam trap A test case is an input **plus its expected output**: give both. For n inputs, normal BVA needs **4n + 1** cases (min, min+, nominal, max−, max), robust BVA **6n + 1** (adds min− and max+): here 5 and 7.

6 Verification & Validation (CMMI Level 3) written

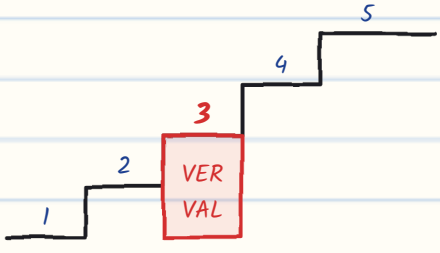
Q9 Verification and validation are two process areas at CMMI level 3. For both: (a) give a definition; (b) describe how you can fulfil these areas in your software testing activities.

Bangladesh Bank, Assistant Maintenance Engineer · 04.02.2023 (BIBM)

(a) Verification (VER) ensures work products meet their specified requirements: "Are we building the product **right**?" Peer reviews, inspections, walkthroughs, static analysis, tests against the spec.

(a) Validation (VAL) shows the product fulfils its intended use in its real environment: "Are we building the **right** product?" UAT, alpha/beta, prototypes, usability tests.

1. **Plan:** pick work products and methods; build a traceability matrix (requirement → test cases)
2. **VER:** review SRS, design and test cases; code review, static analysis; unit, integration and system tests against the spec
3. **VAL:** users set acceptance criteria; UAT in a production-like environment; beta or pilot run
4. **Analyse:** log defects, find root causes, fix, re-test, regression; report coverage, defect density; sign-off



CMMI 1 Initial · 2 Managed · 3 Defined · 4 Quant. Managed · 5 Optimizing

7 The Software Development Life Cycle 10 marks

Q2 Explain the concept of the Software Development Life Cycle (SDLC) and describe its main phases. 10 marks

bank Assistant Programmer, written

Concept SDLC is a structured framework of phases to plan, build, test, deploy and maintain software that meets its requirements on time and within budget. Each phase has set activities and a deliverable that feeds the next; process models (waterfall, incremental, spiral, agile) differ in how they order and repeat the phases.

Phase	Key activities	Output
1 Planning, feasibility	scope, cost, schedule; technical, economic, operational feasibility	feasibility report
2 Requirements	interviews, questionnaires, JAD sessions, prototypes	SRS
3 Design	architecture (HLD), modules (LLD), DB, UI; DFD, ERD, UML	design document
4 Coding	code to standards, version control, unit tests	source code
5 Testing	integration, system and acceptance tests; fix defects	test report
6 Deployment	install, convert data, train users, change over	live system
7 Maintenance	corrective, adaptive, perfective, preventive changes	new versions

Memory trick "**P**lease **R**ead, **D**esign, **C**ode, **T**est, **D**eploy, **M**aintain": P-R-D-C-T-D-M.

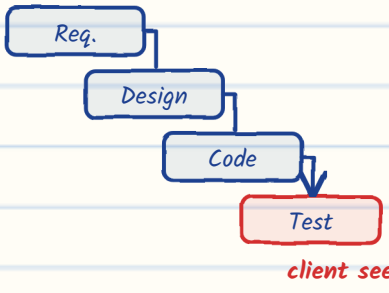
Exam trap Over a product's life, maintenance is the **costliest** phase, and most of it is **perfective** (new features), not bug fixing.

8 System Implementation Stages written

Asked in a recruitment paper: the stages of system development and implementation. 1 preliminary investigation (feasibility) → 2 analysis → 3 design → 4 coding → 5 testing → 6 installation / implementation → 7 maintenance. The implementation stage secures the users' **acceptance** of the system. **Changeover:** direct (cheapest, riskiest) · **parallel** (old and new together: safest, costliest) · pilot (one branch first) · phased (module by module).

9 Waterfall vs Incremental

Point	Waterfall	Incremental
User feedback	only at testing	after each increment
Resources	tied to phases	one increment at a time
Requirements	frozen up front	can evolve
Risk	found late	cut early

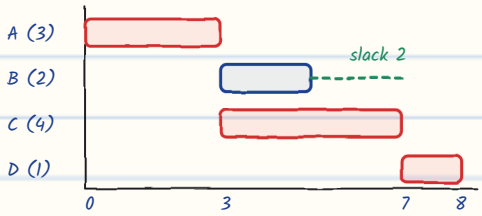


waterfall: one pass, no going back

Asked MCQ **Q40** Key disadvantage of the sequential waterfall model? → Lack of client feedback. Its phases do **not** overlap, and it is documentation-heavy, not lacking. 49th BCS · 2025

10 Project Scheduling & the Gantt Chart MCQ ×2

Asked MCQs ✍ **Q9** Main goal of project scheduling? → ~~allocate resources and set deadlines~~. **Q11** Which is used for project scheduling: pie chart / bar chart / Gantt chart / all? → **Gantt chart**. 49th BCS · 2025



Gantt chart in weeks; red = critical path A–C–D

Gantt chart: tasks as horizontal bars on a time axis, showing start, duration, overlap and progress. It shows **dependencies** poorly, so pair it with a **PERT/CPM network**.

Critical path = longest path = shortest project time; its tasks have zero slack. B and C need A, D needs B and C: A–C–D = 3 + 4 + 1 = **8 weeks**, B may slip 2 weeks.

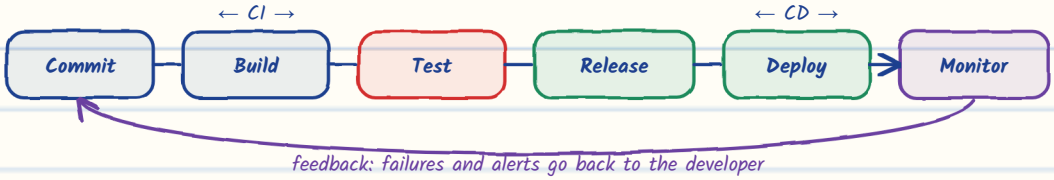
Exam trap A Gantt chart is drawn with bars, but the standard answer is **Gantt chart**; a pie chart shows shares, not time, so "all" is wrong.

11 CI/CD: Stages & Benefits written

Q7 What is the CI/CD development model? Explain its major stages and benefits.

Islamic Bank PLC, SQA Senior Officer (Network) · 14.03.2025

CI (continuous integration): developers merge small changes into the shared main branch several times a day; every merge triggers an automatic build and test run. **CD** = **continuous delivery** (every green build is releasable; going live needs one manual approval) or **continuous deployment** (it goes live automatically).



1. **Source:** a push or pull request starts the pipeline
2. **Build:** compile, fetch dependencies, make an artifact or container image
3. **Test:** unit and integration tests, static analysis, security scans
4. **Release:** deploy to staging; smoke, acceptance, performance tests
5. **Deploy:** to production by rolling, blue-green or canary release
6. **Monitor:** logs, metrics, alerts, rollback

Benefits bugs caught minutes after they are written · small, low-risk releases · faster delivery · repeatable builds, fewer manual errors · quick rollback · constant feedback. Tools: Git, Jenkins, GitHub Actions, GitLab CI, Docker, Kubernetes.

Git: commit vs push **commit** saves a snapshot in your **local** repository (works offline); **push** uploads local commits to the **remote** so the team and CI see them. bank Assistant Programmer, QS

Memory trick **add** → stage · **commit** → my desk · **push** → the team's shelf · **pull** = fetch + merge. CI = "merge often, test always"; CD = "always ready to ship".

12 Coupling & Cohesion written

Q3 Explain the concepts of Coupling and Cohesion with examples.

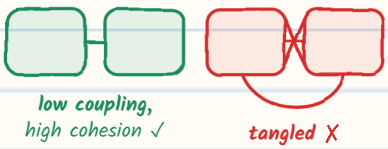
Bangladesh Satellite Company PLC, Assistant Engineer (CSE) · 23.08.2025

Coupling = how much modules depend on each other; aim low, so a change does not ripple. **Cohesion** = how strongly the parts inside one module serve one task; aim high, so the module is easy to understand, test and reuse.

Coupling, best → worst	Example	Cohesion, best → worst	Example
Data	area(w, h)	Functional	computeTax(): one job
Stamp	whole record, one field used	Sequential	read → validate → save
Control	print(rep, flag)	Communicational	print + update one record
External	share a file format	Procedural	steps in order, other data
Common	same global variable	Temporal	init() at start-up
Content	edits another's internals	Logical	doIO(type) by flag
		Coincidental	Utils of unrelated code

Memory trick Coupling: "**D**o **S**weet **C**hildren **E**at **C**old **C**akes?" Cohesion: "**F**un **S**equels **C**ommunicate **P**lots **T**hat **L**ook **C**oincidental".

Exam trap Good design = **low coupling, high cohesion**. "High coupling" is never a design principle.



13 Code Smells: Examples & Fixes written ×4

Q11 Give examples of the following code smells: (a) Feature Envy (b) Dead Code (c) Duplicate Code (d) Shotgun Surgery.

asked in a recruitment paper

(a) Feature Envy: a method uses another object's data more than its own.

```
class ShippingCalc {
 double cost(Order o) {
  return o.getWeight()
       * o.getZone().getRate(); } }
```

Fix: **Move Method** → Order.shippingCost().

(b) Dead Code: never runs, or its result is never used.

```
double price(double p) {
  double old = p * 0.95; // never used
  if (LEGACY_V1) { ... } // always false
  return p * 0.90; }
```

Fix: delete it; version control keeps the history.

(c) Duplicate Code: the same logic pasted in several places.

```
// in register() AND updateProfile()
if (email == null
    || !email.contains("@"))
  throw new IllegalArgumentException();
```

Fix: **Extract Method** validateEmail().

(d) Shotgun Surgery: one change forces small edits in many classes.

```
// VAT 0.15 typed into Cart, Invoice,
// Receipt and Report: new rate = 4 edits
```

Fix: one TaxPolicy class or constant (Move Field).

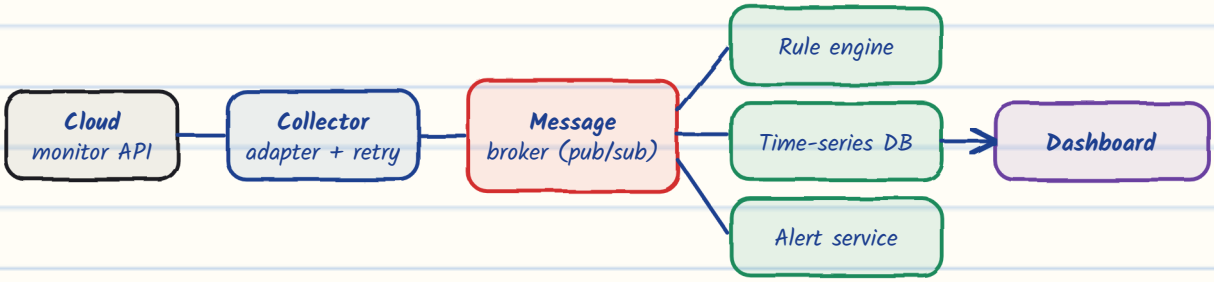
Exam trap **Shotgun surgery** = one change → many classes. **Divergent change** is the opposite: one class changed for many different reasons.

14 Architecture & Patterns: a Monitoring System written

Q2 A third-party cloud service is used for continuous system monitoring through APIs. (a) Which software architecture would you recommend? (b) Which design patterns would be appropriate? (c) Explain why you selected them.

Bangladesh Satellite Company PLC, Assistant Engineer (CSE) · 21.08.2026

(a) Architecture: an event-driven architecture of small services (microservices; or modules of one app for a small team), with a **message broker** in the middle and an **adapter layer** facing the third-party API.



poll the API or receive webhooks → publish events → independent subscribers

(b) Pattern	Where it is used
Adapter / Facade	wrap each vendor API behind our own MetricsSource interface
Observer / Publish–Subscribe	rule engine, alerting and dashboard subscribe to metric events
Circuit Breaker + Retry	vendor outages, timeouts, rate limits (exponential backoff)
Strategy · Factory	swappable alert rules and channels · a collector per provider

(c) Justification ① **Loose coupling to the vendor:** the adapter isolates API changes; a new provider touches one class. ② **Scalability:** collectors and processors scale separately; the broker absorbs bursts. ③ **Reliability:** the circuit breaker and the queue stop one failure from cascading, and no event is lost. ④ **Real-time:** events are pushed to subscribers at once. ⑤ **Extensibility:** a new alert channel is one more subscriber or strategy (open–closed principle). ⑥ **Testability:** every part can be tested against a mock API.

Exam trap State the trade-off: microservices add deployment and monitoring overhead; for a small team a modular monolith with the same patterns is a valid answer. The justification earns more marks than the label.

15 User Interface Design MCQ

Asked MCQ ✍️ **Q72** Which principle is followed in UI design: high coupling / user-centred design / sequential processing / cost optimization? → **User-centred design**. 49th BCS · 2025

User-centred design design around real users' goals and tasks: involve users early, test usability with them, iterate. Golden rules: consistency, feedback, error prevention, easy undo, user in control, low memory load.

16 COCOMO: Development Time

MCQ math

Q96 An organic software has 32 KLOC. The nominal development time (in months) using the COCOMO model is: (a) 6 (b) 7 (c) 14 (d) 21

49th BCS ICT paper 2 · 2025

Basic COCOMO (Boehm, 1981)

Effort $E = a(KLOC)^b$ person-months

Time $T = c(E)^d$ months

Staff = E / T · productivity = LOC / E

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Solution

1. $32^{1.05} = 38.05$
2. $E = 2.4 \times 38.05 = 91.3$ PM
3. $91.3^{0.38} = 5.56$
4. $T = 2.5 \times 5.56 = 13.9 \approx$ **14 months** → **(c)**
5. Staff = 91.3 / 13.9 ≈ 6.6 ≈ 7 people; productivity ≈ 350 LOC/PM
6. Same 32 KLOC: semi-detached 145.5 PM, embedded 230.4 PM, both ≈ 14.3 months

Exam trap Intermediate COCOMO uses a = **3.2** for organic (then × EAF from 15 cost drivers). Putting 3.2 into the basic formula gives E ≈ 121.8 PM and T ≈ **15.5** months, not 14. Option (b) 7 is the **staff** figure, not months.

17 Risk Analysis Techniques

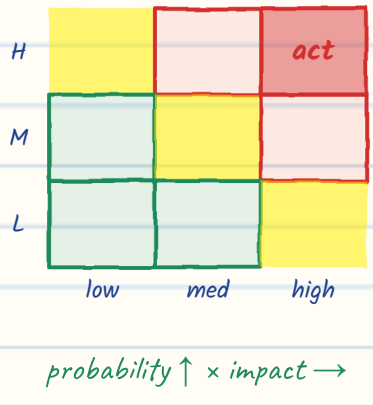
written

Q9(a) Describe the different techniques used in risk analysis of software development.

38th BCS Written · 2018

Risk management = **identify** → **analyse** → **prioritise** → **plan** → **monitor**. The analysis techniques:

1. **Identification:** risk checklists, brainstorming, Delphi (anonymous experts), interviews, SWOT, assumption analysis, past-project lessons
2. **Categorising:** project, product, business risks; or five impact areas: new technology, requirements, architecture, performance, organisation
3. **Qualitative:** rate probability and impact (negligible → catastrophic) on a probability–impact matrix
4. **Quantitative:** risk exposure $RE = P \times L$; decision trees (expected value), sensitivity analysis, Monte Carlo simulation
5. **Prioritise & respond:** rank by RE; avoid, mitigate, transfer or accept (RMMM plan)



Worked example	P	Loss L	RE = P × L	Rank
requirements keep changing	0.6	Tk 5 lakh	Tk 3.0 lakh	1
key developer leaves	0.3	Tk 6 lakh	Tk 1.8 lakh	2
third-party API changes	0.5	Tk 2 lakh	Tk 1.0 lakh	3

Exam trap The question asks for **techniques**. Listing only risk categories (technology, requirements …) earns part marks; add identification methods, the matrix and RE = P × L.

18 Viva Rapid-Fire

bank IT viva

Q. What is an API?
→ Application Programming Interface: a contract (endpoints, formats, rules) that lets one program use another's services or data without knowing its internals.

Q. How does an API work?
→ The client sends a request (HTTP method, URL, headers with an auth token, JSON body) → the API authenticates, runs the logic, reads the DB → returns a status code (200, 201, 400, 401, 404, 500) and JSON. Styles: REST, SOAP, GraphQL, gRPC.

Q. What is an Information System?
→ people, hardware, software, data and procedures (plus network) that collect, process, store and distribute information.

Q. Role of IS in an organisation?
→ runs daily transactions (TPS), supports decisions (MIS, DSS, ESS), communication, records and compliance, competitive advantage.

Q. Management software? Examples?
→ software to plan, run and control resources and processes: ERP (SAP, Oracle, Microsoft Dynamics 365, Odoo), Tally (accounting), CRM, HRM/payroll, Jira; in banks, core banking (Temenos T24, Finacle).

Q. Do you know app development?
→ Android: Kotlin/Java; iOS: Swift; cross-platform: Flutter (Dart), React Native; the app talks to the server through REST APIs. Name an app you built.

Q. What is ERP? Why is it used?
→ Enterprise Resource Planning: one integrated suite on a **single database** for finance, HR, procurement, inventory, sales, production. Gives one source of truth, no double entry, automated workflows, real-time reports, an audit trail.

Q. What is MRP? MRP vs ERP?
→ Material Requirements Planning: from the production schedule, bill of materials and stock it decides **what material, how much, when**. MRP = manufacturing materials only; MRP II added capacity and finance; **ERP = the whole enterprise**.

Q. System vs solution?
→ **System**: interacting components doing one function (a core banking system). **Solution**: the full answer to a business problem: one or more systems plus process change, integration, training, support.

Q. How is ISO applied in software?
→ ISO 9001 (quality management), ISO/IEC 12207 (life-cycle processes), ISO/IEC 25010 (product quality), ISO/IEC/IEEE 29119 (testing), ISO/IEC 27001 (security); through defined processes and certification audits.

Q. Levels and methods of testing? Acceptance testing?
→ unit, integration, system, acceptance (sheet 2); black, white, grey box (sheet 3). Acceptance = users confirm business fit before go-live.

19 All MCQs Asked, at a Glance

49th BCS · 2025

Q	Asked	Answer	Why
9	goal of scheduling	resources and deadlines	who does what, when
11	scheduling tool	Gantt chart	tasks on a time axis
21	NOT a white-box focus	user perspective	that is black-box
40	waterfall's drawback	no client feedback	client sees it at the end
72	UI design principle	user-centred design	design around users
86	"structural testing"	white box	tests code structure
96	organic 32 KLOC time	**14 months**	$2.5 \times 91.3^{0.38}$

Night-Before-the-Exam Sheet

Formulas & numbers

$E = a \cdot KLOC^b$ · $T = 2.5 \cdot E^d$ · staff = E/T

embedded 3.6/1.20/0.32 · intermediate a = 3.2

V(G) = E − N + 2 = decisions + 1

RE = probability × loss

CMMI: 1 Initial · 2 Managed · 3 Defined

organic 2.4/1.05/0.38 · semi 3.0/1.12/0.35

32 KLOC organic → 91 PM, 14 months, ≈ 7 people

BVA: 4n + 1 cases · robust: 6n + 1

critical path = longest path, zero slack

4 Quantitatively Managed · 5 Optimizing

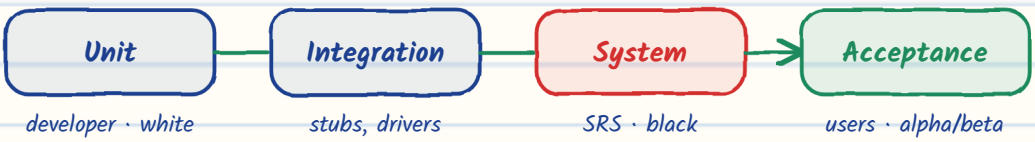
→ One-liners that were asked

1. Levels: unit → integration → system → acceptance
2. Unit = developer; UAT = users (alpha, beta)
3. Regression = both; load = non-functional
4. Smoke = shallow check of a new build
5. White box = structural = glass box
6. Black box = behavioural, the user's view
7. Grey box = partial knowledge (pen-testing)
8. Verification = product right; validation = right product
9. VER and VAL: CMMI level-3 process areas
10. Waterfall flaw: no client feedback till the end
11. Changeover: direct, parallel, pilot, phased
12. CI = merge often, auto build/test; CD = always releasable
13. commit = local, push = remote
14. Low coupling (data best), high cohesion (functional best)
15. UI design = user-centred design
16. ERP = one database for every department

Top traps regression is "both" · a test case needs its expected output · BVA values sit on and just outside each edge · intermediate a = 3.2 gives 15.5 months, not 14 · 7 in the COCOMO MCQ is staff, not months · waterfall phases do not overlap · shotgun surgery ≠ divergent change · risk "techniques" ≠ risk categories

All tricks together V-model: each test level checks the document opposite it · Black = Behaviour, White = Wiring · P-R-D-C-T-D-M for SDLC · add → stage, commit → my desk, push → team shelf · "Do Sweet Children Eat Cold Cakes?" (coupling) · "Fun Sequels Communicate Plots That Look Coincidental" (cohesion)

small → whole → business sign-off



the four levels in one line

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. Basic COCOMO effort for 32 KLOC, organic:
(a) 46 PM (b) 91 PM
(c) 122 PM (d) 146 PM

2. Which testing type is non-functional?
(a) Smoke (b) Unit
(c) Integration (d) Load

3. "Are we building the right product?" is:
(a) Validation (b) Verification
(c) Regression (d) Inspection

4. The most desirable coupling is:
(a) Content (b) Common
(c) Data (d) Control

5. Re-running old tests after a bug fix is:
(a) Smoke (b) Regression
(c) Alpha (d) Stress

6. A flow graph has 9 edges, 7 nodes. V(G) =
(a) 4 (b) 2
(c) 3 (d) 16

7. Input valid from 1 to 100: equivalence classes =
(a) 1 (b) 2
(c) 4 (d) 3

8. One change forces edits in many classes:
(a) Feature envy (b) Dead code
(c) Shotgun surgery (d) Duplicate code

9. Pattern that wraps a third-party API behind your own interface:
(a) Adapter (b) Singleton
(c) Observer (d) Strategy

10. VER and VAL are process areas at CMMI level:
(a) 2 (b) 5
(c) 4 (d) 3

Answer key → 1-b · 2-d · 3-a · 4-c · 5-b · 6-a · 7-d · 8-c · 9-a · 10-d

Written question Using basic COCOMO, estimate the effort, development time, average staff and productivity of a 50 KLOC **semi-detached** project. How would the effort change if it were organic? 8 marks

1. Semi-detached: $a = 3.0$, $b = 1.12$, $c = 2.5$, $d = 0.35$; $E = a \cdot KLOC^b$, $T = c \cdot E^d$ 1
2. $50^{1.12} = 79.96 \rightarrow E = 3.0 \times 79.96 =$ **239.9 PM** 2
3. $239.9^{0.35} = 6.81 \rightarrow T = 2.5 \times 6.81 =$ **17.0 months** 2
4. Staff = 239.9 / 17.0 ≈ **14 people**; productivity = 50 000 / 239.9 ≈ **208 LOC/PM** 2
5. Organic: $E = 2.4 \times 50^{1.05} = 145.9$ PM, $T = 16.6$ months: about **40% less effort** for nearly the same time 1

→ Before the exam ✓

- ✓ Can name any testing scenario: functional / non-functional / both
- ✓ Can write EC + BVA cases with expected outputs
- ✓ Can draw a flow graph and compute V(G)
- ✓ Can write the SDLC phases with outputs (10 marks)
- ✓ Can do COCOMO in all three modes

Also practise (written):
① V&V for CMMI level 3
② coupling & cohesion with code
③ an example of each code smell
④ CI/CD stages and benefits
⑤ risk analysis techniques